Practical

2a - C++ threads and locks

```
#include <thread>
void Foo(int a, int b) {...}
// the thread will execute foo with arguments 1, 2
std::thread t(Foo, 1, 2);
// [x, &y]: captured by value and reference
std::thread t([x, &y]() -> void { y=x*42; });
t.join(); // remember to join the thread!
```

Passing a reference to std::thread:

```
#include <functional> // std::ref
void ThreadBody(std::string by_value, std::string& by_reference) {...}
std::string hello = "Hello";
std::string world = "World";
// use std::ref to pass thread argument by reference
// use std::cref for const references
std::thread t(ThreadBody, hello, std::ref(world));
```

Vector of threads:

```
std::vector<std::thread> threads;
for (size_t i = 0; i < 16; i++) { // launch a bunch of threads
 threads.push_back(std::thread(ThreadBody, a, b, c));
 // alternative: construct an object in-place at end of vector
 // type inference determines that std::thread constructor should be
invoked with ThreadBody, a, b, c
 threads.emplace_back(ThreadBody, a, b, c); }
// wait for them to finish
for (auto& thread : threads){ thread.join(); }
```

Mutex:

```
#include <mutex>
std::mutex mutex_; mutex_.lock(); mutex_.unlock();
```

RAII: Resource Acquisition is Initialization

Also known as: Constructor Acquires, Destructor Release. OR Scope-based Resource Management (Only applies to stack-allocated data) Widely-used pattern for resource management in C++. Encourages thinking about ownership and lifetimes of objects.

```
{ // mutex acquired when scoped_lock is constructed
 std::scoped_lock<std::mtx> lock(mutex_);
 ... } // mutex released when scoped_lock goes out of scope and is
(implicitly) destroyed
```

Race conditions

A **data race** occurs when:

- Distinct threads **access** a memory location
- At least one of the accesses **modifies** the location
- At least one of the accesses is **non-atomic**
- The accesses are **not ordered** by synchronization

The behaviour of C++ program P on input I is **undefined** if a data race can occur when P is executed on I (the program is allowed to do **anything**).

Compiler Optimization

The compiler optimizes your program **on the assumption that there is no undefined behaviour.** An optimization that assumes no undefined behaviour is: Sound when there is no undefined behaviour (by definition). Sound when there is undefined behaviour, because in that case **anything goes**.

```
static void Foo(int& x) { x = 1; }
static void Bar(int& x) { while (x==0) {} }
int main() { int x = 0; // then initialize and join t1 and t2 and return 0}
```

Keep r in a register – avoid tons of loads from memory.

x is a **non-atomic variable.** If another thread modified x concurrently, that would be a data race**. A data race is an undefined behaviour** (Assume don't happen -> no one else modifies x –> optimize)

```
static void Bar(int& x){int r = x; while (r==0){}}
```

Solution: add volatile will forces compiler to load x from memory each time it is accessed. (doesn't solve data race)

Eliminate side-effect free infinite loop.

- This is **an infinite loop without a side effect**
- That's an undefined behaviour (Assume don't happen -> this code doesn't get executed -> optimize)

```
static void Bar(int& x) { // Nothing }
```

Solution: make x atomic so that thread must read the most up-to-date value of x.

A data race occurs even when both threads write the same value. E.g. When T1 and T2 both store 42 to x. (despite both threads storing 42). When T1 store 42 to x, T2 load from x, and x = 42 originally. (despite existing value of x being 42).

std::scoped_lock: Constructed with one or more mutexes**.** Locks all mutexes on construction**.** Unlocks all mutexes on destruction**. Does not support:** Deferred locking - delaying locking until after construction. Early unlocking - unlocking before destruction. Ownership transfer - using std::move to change who owns the associated mutexes.

std::unique_lock: Constructed with one mutex. Locks mutex on construction (default), or locking can be deferred. Allows unlocking and relocking. Unlocks mutex on destruction if still held. Supports transfer of ownership to a distinct unique owner via std::move. Does not support management of multiple mutexes.

std::scoped_lock is more efficient: use it unless you require the features it does not support

Condition variables: Use std::condition_variable to allow threads to wait on one another.

```
#include <condition_variable>
// Declare a condition variable; usually a class member
std::condition_variable condition_;
// Notify one or all threads waiting on the condition variable
// notify_one for 1 wait predicate; notify_all for more predicates
condition_.notify_one(); condition_.notify_all();
```

To wait on a condition variable:

- Associate a mutex with a condition variable (Multiple conditions might share the same mutex)
- Acquire the mutex via std::unique_lock
- Call wait, passing the lock and a predicate (Zero-argument boolean function; always a lambda in practice)

```
std::unique_lock<std::mutex> lock(mutex_);
// wait until the predicate is true
condition_.wait(lock, [...]() -> bool { return ...; });
// [...] : Capture relevant state to evaluate the predicate
```

wait(lock, predicate): Requires that **lock is held by the thread that calls wait.** Repeats the following: Return if predicate holds -> Release lock -> Block until another thread signals (via notify_one or notify_all) -> ... -> ...

```
class FifoQueue {
 // ...
 void Enq(const int value) {
   std::unique_lock<std::mutex> lock(mutex_);
   not_full_.wait(lock, [this]() -> bool { return
            count_ < elements_.size(); });
   elements_[tail_] = value;
   tail_ = (tail_ + 1) % elements_.size();
   count_++;
   not_empty_.notify_one();
 }
 int Deq() {
   std::unique_lock<std::mutex> lock(mutex_);
   not_empty_.wait(lock, [this]() -> bool { return
            count_ != 0; });
   int result = elements_[head_];
   head_ = (head_ + 1) % elements_.size();
   count_--;
   not_full_.notify_one();
   return result;
 }
 // ...
};
```

Reader-writer locks: std::shared_mutex, std::shared_lock, from <shared_mutex>

Recursive (re-entrant) mutexes: std::recursive_mutex, from <mutex>

Deferred locking:

```
// std::defer_lock: Tag, indicating that mutex should not be locked on
construction
std::unique_lock<std::mutex> l(m, std::defer_lock);
if (...) { // Mutex can later be acquired if needed. RAII still means that
mutex is released on lock destruction
 l.lock();}
```

2b - C++ atomics

Atomics

- Allow well-defined race conditions
- Allow fine-grained synchronization between threads

Read-modify-write (RMW) operations

An RMW operation on an atomic variable: Reads the old value of the variable. Writes a new value to the variable. The new value is either a completely new value or a function of the old value

All this is performed as one indivisible step - other threads cannot observe the effects of an RMW operation midway through.

std::atomic<T>:

- Declares an atomic variable of type T
- Most common use cases: boolean, integral and pointer types

```
std::atomic<MyStruct *> p(&my_struct);
```

Operations on std::atomic<T>:(All operations are atomic)

store(x): store value x of type T

load(): yields value of type T

Read-modify-write (RMW) operations:

exchange(x): store x and return old value

compare_exchange_strong(expected, desired)

compare_exchange_weak(expected, desired): allowed to fail spuriously, i.e. behave as if expected != desired even when expected == desired. Failure should occur with low probability.

Compare_Exchange: Consider: x.compare_exchange(e, d);

The value of x and e are compared.

- If equal: d is stored to x and true is returned.
- If not equal: x is stored to e, x is left unchanged and false is returned.

Comparing and exchanging combined into one indivisible operation (RMW). If two threads try to concurrently compare-exchange x from a to b (a != b), at most one will succeed.

Operations on std::atomic<integral_type>: (All RMW)

fetch_add(x): Replace value with value + x, return old value

fetch_sub(x), fetch_and(x), fetch_or(x), fetch_xor(x) (similar)

Operations on std::atomic<integral_type>: (All RMW)

fetch_add(x): Replace value with value + x, return old value

fetch_sub(x), fetch_and(x), fetch_or(x), fetch_xor(x) (similar)

Relaxed memory order: std::memory_order_relaxed

- The weakest memory ordering
- Only guarantee: sequential consistency **per location, no inter-thread synchronizations!**
- Gives the compiler more flexibility to reorder instructions - useful for optimisations (which means **operations on different locations can be re-ordered**)
- Avoids the need to issue expensive memory barriers
- Useful when you really don't care in which order things happen
- Parallel find: when there are multiple occurrences we don't care which one is reported
- Sequential consistency facilitates message passing. Relaxed atomics cannot guarantee message passing.

Release-acquire consistency: made for message passing

T1	T2
// Prepare data *data = 42;* *// Notify other thread* *flag.**store**(true, **release**);*	*// Wait for data to be ready* *while(!flag.**load**(**acquire**)) {}* *// Spin }* *// Use data* *Foo(data); // guarantee 42*

- **Acquire (load)** stops any operations **before** it to be reordered to after it ("Something has been done"). (force thread to flush buffer)
- **Release (store)** stops any operations **after** it to be reordered to before it. ("I want to receive message from another thread") (get all information flushed by previous threads)

May require **fewer** memory barriers than SC, but **more** memory barriers than relaxed - depends on architecture.

Memory Ordering for RMW:

x.fetch_add(1);	Sequentially consistent load and store
*x.fetch_add(1, **std::memory_order_relaxed**);*	Relaxed load and store
*x.fetch_add(1, **std::memory_order_release**);*	Relaxed load, release store
*x.fetch_add(1, **std::memory_order_acquire**);*	Acquire load, relaxed store
*x.fetch_add(1, **std::memory_order_acq_rel**);*	Acquire load, release store

3a - Spinlocks

Spinlocks

Useful when lock delay is expected to be short: Short critical sections and/or Low contention. Avoids making OS system calls.

Test-and-set spinlock

Test-and-set (TAS): Performed atomically:

```
struct tas_lock {
 std::atomic<bool> latch_ = {false};
 void lock() { while (latch_.exchange(true,
          std::acquire)); }
 void unlock() { latch_.store(false,
          std::release); }
};
```

Problem: **Cache thrashing:** When multiple cores spin on a shared lock variable, each core repeatedly reads the lock variable. When one core execute TAS on the lock variable, the memory location of the shared lock variable becomes invalid on other cores (i.e. this core invalidates all other cores). Creates excessive traffic on memory bus.

Local spinning: Reduce bus traffic

- Repeatedly load cached copy of lock state
- Only do TAS when the lock is seen to be available

TAS is still needed: another thread might get there first!

```
struct ttas_lock {
 ...
 void lock() {
   while (latch_.exchange(true, std::acquire)) {
     while (latch_.load(std::relaxed));
   }
 }
 ...
};
```

Bus traffic still a problem: Many threads locally spinning. Threads detect lock is free almost simultaneously. Will have a TAS fight.

Active backoff: On each iteration of local spinning, do nothing for a while!

NOTE: use `volatile`-for loop for this.

Passive backoff: use one of these instructions instead of volatile loop: Processor instructions for hinting that program is spinning: x86: pause. ARM: YIELD.

Allows the processor to "do nothing" more efficiently. Reduces energy consumption. Does not induce OS context switch.

NOTE: use `_mm_pause()` for this.

- Short backoff: Little time wasted "doing nothing". Threads may attempt TAS at uncomfortably similar times.
- Long backoff: More chance threads will diverge and attempt TAS at different times. More time spent "doing nothing" – inefficient.

Exponential backoff:

- Double the amount of time spent "do nothing" every iteration of local spinning
- Start from some small initial value
- Stop doubling when a maximum value is reached
- Widely-used in networking to avoid network congestion when re-transmitting data

Exponential backoff is not used for generic spinlocks in practice: backoff parameters require application-specific tuning to work well. Backoff can underutilise critical section when lock is contended

- Threads back off when they notice contention
- When a thread releases the lock, there may be some delay before any thead acquires it
- More pronounced 显著的 at higher contention

Spinlocks and fairness:

If contention is high, a thread may repeatedly fail to get the lock

Negative consequence of unfairness: starvation of threads Positive consequence of unfairness:

- Suppose lock grants access to shared data structure
- Thread accesses data structure => relevant data is in cache
- Thread gets access again directly => data still in cache

Tension between fairness and performance

Ticket lock

When a thread wants the lock, it obtains a ticket.

A thread holds the lock when its ticket number is being served.

When a thread releases a lock, the next ticket number gets served.

```
struct ticket_lock {
 std::atomic<size_t> next_ = {0};
 std::atomic<size_t> curr_ = {0};
 void lock() {
   auto ticket = next_.fetch_add(1, std::relaxed);
   while (curr_.load(std::acquire) != ticket) {
     _mm_pause();
   }
 }
 void unlock() {
   auto current = curr_.load(std::relaxed);
   curr_.store(current + 1, std::release);
 }
};
```

NOTE: performs worse than spinlock, but more fair.

3b - Futexes and Hybrid Locks

Futexes and Hybrid Locks

Spinlocks vs. sleeping locks: System calls are expensive: going to sleep is expensive. Spinning involve doing work all the time: being awake is expensive.

Spinlock: Threads intensively check whether lock is free. Checks performed entirely in user space: no system calls required. Great when there is low contention, or short critical sections. Prevents CPU from doing other useful work when contention is high.

Sleeping lock: Thread that doesn't manage to get lock asks OS to put it to sleep. OS wakes thread up when lock becomes free: requires system calls. Good idea to sleep if lock won't be free for a while: CPU can do other things. Bad idea to sleep if lock is just about to become free.

Futex: fast userspace mutex: futex is a Linux system call - exposes functionality offered by the kernel (so not userspace). But futex works on userspace data, and can be used to implement synchronisation primitives that execute primarily in userspace. futex is very flexible and can implement more than just mutexes.

```
// 'SYS_futex' is a SYSCALL with many OPERATIONS
int futex(uint32_t *p, int op, uint32_t v, const
        struct timespec *timeout, uint32_t *p2,
        uint32_t v3) {
 return syscall(SYS_futex, p, op, v, timeout, p2,
        v3);
}
// 'FUTEX_WAIT' operation:
// - IF '*p != v' THEN sets 'errno=EAGAIN' and
        returns '-1' immediately
// - OTHERWISE puts thread to sleep & adds it to
        KERNEL SPACE wait queue associated with
        'p'; returns '0' when woken up
int futex_wait(uint32_t *p, uint32_t v) {
 return futex(p, FUTEX_WAIT, v, nullptr, nullptr,
        0);
}
// 'FUTEX_WAKE' operation:
// - Wake up at most 'wake_count' threads in
        'p'-associated wait queue; returns
        actual number of woken up threads
// - 'wake_count' most commonly '1' or 'INT_MAX'
int futex_wake(uint32_t *p, uint32_t wake_count) {
 return futex(p, FUTEX_WAKE, wake_count, NULL,
        NULL, 0);
}
```

Simple implementation

```
struct mutex_simple {
 const int kFree = 0; const int kLocked = 1;
 std::atomic<int> state_ = {kFree};
 void lock() {
   while (state_.exchange(kLocked) == kLocked) {
     futex_wait(&state, kLocked);
   }
 }
 void unlock() {
   state_.store(kFree);
   futex_wake(&state, 1); // wake up 1 waiter
 }
};
```

Pointer p passed to FUTEX_WAIT and FUTEX_WAKE can be the address of any int-sized piece of data

The OS doesn't need to know about this data before you call futex

The futex system call doesn't care what meaning you ascribe to the values of *p

We store futex waiter queues in kernel space, when a thread call FUTEX_WAIT, we create new wait queue keyed by the address p. When all waiters are woken, queue memory is freed.

Barging: a sleep thread is woken, but another new thread acquire the lock before the woken thread.

Problem: wasteful to call FUTEX_WAKE when there are no waiters

Smarter futex-based mutex design

Three state values:

- 0: mutex is available
- 1: mutex is locked; the thread that locked it thinks there are no waiters
- 2: mutex is locked; the thread that locked it thinks there might be waiters

```
class mutex_smarter {
 const int kFree = 0;
 const int kLockedNoWaiters = 1;
 const int kLockedWaiters = 2;
 void lock() {
   int old = cmpxchg(kFree, kLockedNoWaiters);
   if (old == kFree) return; // got lock w/out
          contention

   do {
     if (old == kLockedWaiters ||
          cmpxchg(kLockedNoWaiters,
          kLockedWaiters) != kFree) {
       futex_wait(&state_, kLockedWaiters);
     }
     old = cmpxchg(kFree, kLockedWaiters);
   } while (old != kFree);
 }
 void unlock() {
   if (state_.exchange(kFree) == kLockedWaiters) {
     futex_wake(&state_, 1);
   }
 }
private:
 std::atomic<int> state_ = {kFree};
 int cmpxchg(int expected, int desired) {
   int e = expected; // mutable stack variable
   state_.compare_exchange_strong(e, desired);
   return e;
 }
};
```

- Zero kernel memory used when there are no waiters
- Waiter queues created on demand: map from address of user-space integer to queue
- Waiter queue disappears when last waiter is woken, reappears if there are more waiters in the future
- The futex system call doesn't care how the userspace code treats the value of the user-space integer

4a - Concurrency in Haskell

Concurrency in Haskell

Haskell's *MVar* is a key building block for concurrency As we shall see, it allows a mixture of: shared memory concurrency and message passing concurrency.

```
import Control.Concurrent
forkIO :: IO () -> IO ThreadId
```

Kicks off the computation asynchronously in a new thread; immediately returns the new thread's id. forkIO causes a thread to run asynchronously. When main ends, the program is terminated even if threads are still running.

MVar a - mutable variable storing a value of type a

An MVar is either empty (holds no value) or full (holds a value of type a).

Create a new MVar with a given value:

```
newMVar :: a -> IO (MVar a)
```

Create a new empty MVar:

```
newEmptyMVar :: IO (MVar a)
```

Take value from an MVar:

```
takeMVar :: MVar a -> IO a
```

Blocks until the MVar is full, leaves the MVar empty. Yields the value that was in the MVar.

Put value into an MVar:

```
putMVar :: MVar a -> a -> IO ()
```

Blocks until the MVar is empty. Leaves the MVar full, containing the given value.

Read current value from MVar:

```
readMVar :: MVar a -> IO a
```

Blocks until the MVar is full. Yields the current value of the MVar, leaving it full.

Joinging threads using an MVar:

```
thread :: ... -> MVar () -> IO ()
thread ... handle = do
 -- Indicate that this thread has finished
 putMVar handle ()
main = do
 handle <- newEmptyMVar
 forkIO (thread ... handle)
 -- Wait for thread to finish: blocks until there is something to take
 takeMVar handle
```

Getting a value from a thread join:

```
thread :: ... -> MVar Int -> IO ()
thread ... handle = do ...
 result <- ...
 -- Indicate that this thread has finished, providing a result
 putMVar handle result
main = do
 handle <- newEmptyMVar
 forkIO (thread ... handle)
 -- Wait for thread to finish and get its result
 result <- takeMVar handle
```

Using MVars for mutal exclusion:

```
thread :: ... -> MVar () -> ... -> IO ()
thread ... mutex ... = do
 putMVar mutex ()
 -- Critical section
 takeMVar mutex
main = do
 mutex <- newEmptyMVar
 -- Launch two threads, with access to a common mutex
 forkIO (thread ... mutex ...)
 forkIO (thread ... mutex ...)
```

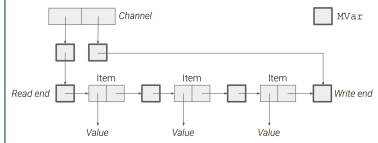
Replicate monad with

```
replicateM :: Monad m => Int -> m a -> m [a]
```

e.g. take from MVar 10 times in a row

```
values <- replicateM 10 (takeMVar v) :: IO [Int]
```

Unbounded channels




```
data Channel a = Channel (MVar (Stream a)) (MVar
        (Stream a))
type Stream a = MVar (Item a)
data Item a = Item a (Stream a)

newChannel :: IO (Channel a)
newChannel = do
 emptyStream <- newEmptyMVar
 readEnd <- newMVar emptyStream
 writeEnd <- newMVar emptyStream
 return (Channel readEnd writeEnd)
readChannel :: Channel a -> IO a
readChannel (Channel readEnd _) = do
 readEndStream <- takeMVar readEnd
 (Item value remainder) <- takeMVar readEndStream
 putMVar readEnd remainder
 return value
writeChannel :: Channel a -> a -> IO ()
writeChannel (Channel _ writeEnd) value = do
 newEmptyStream <- newEmptyMVar
 writeEndStream <- takeMVar writeEnd
 putMVar writeEndStream (Item value
        newEmptyStream)
 putMVar writeEnd newEmptyStream
```

IORef - mutable atomic reference

```
newIORef :: a -> IO (IORef a)
readIORef :: IORef a -> IO a
atomicWriteIORef :: IORef a -> a -> IO ()
```

AtomicCounter

```
newCounter :: Int -> IO AtomicCounter
incrCounter :: Int -> AtomicCounter -> IO Int
incrCounter_ :: Int -> AtomicCounter -> IO ()
readCounter :: AtomicCounter -> IO Int
```

4b - Communication in Rust

```
// can only capture 'static refs
let h = thread::spawn(|| -> T {...});
h.join().unwrap(); // remember to join handle

// attach condvar to data with this
let p: Arc<(Mutex<T>, Condvar)> = ...;
let (m, cv) = &*p; // unpack mutex & condvar
cv.notify_one/all(); // condvar notify
let mut guard = m.lock().unwrap();
let val = &mut *guard; // ASIDE: must deref
while /*actual cond.*/ { // wait for condvar
 guard = cv.wait(guard).unwrap();
}
```

5 - Dynamic Data Race Detection

Vector clock-based race detection

A thread's **logical clock**: a non-negative integer. A thread's logical clock value **increases** each time the thread releases a mutex. Releasing a mutex indicates transitioning to next phase of concurrent program.

A **vector clock** is a tuple of N logical clocks, one per thread: $(c_0, c_1, \ldots, c_{N-1})$. Equivalently: vector clock is a mapping from Threads to natural numbers. If V is a vector clock, $V(t)$ denotes logical clock of thread t. Set of all vector clocks: VC

Bottom vector clock: $\bot = (0, 0, \ldots, 0)$. Every thread has logical clock 0

Partial order on vector clocks: $V_1 \sqsubseteq V_2$ if and only if $\forall t. V_1(t) \leq V_2(t)$. One vector clock is "less-than or equal to" another vector clock if logical clocks are pointwise less-than or equal.

Join of vector clocks: $V_1 \sqcup V_2$ = pointwise maximum of V_1, V_2

Increment: $inc_t(V) = V[t \mapsto V(t) + 1]$. Same as V, but logical clock for t is incremented.

Algorithm state: (C, L, R, W):

- $C : Threads \to VC$. $C_t = C(t)$. C_t represents what thread t knows about the logical clocks of all threads. If I am thread t, then: $C_t(t)$ is my logical clock. It will always be **positive**. For $u \neq t, C_t(u) = z$ means "I know that thread u's logical clock is at least z". When t **acquires a lock**, t gets information about the logical clocks of threads who previously held the lock.
- $L : Locks \to VC$. $L_m = L(m)$. L_m represents the value of the vector clock associated with the last thread that released m, when the release occurred. For a lock m and thread t: $L_m(t) = z$ means that the last thread to release m knew that thread t's logical clock was at least z when it released the mutex.
- $R : Locations \to VC$. $R_x = R(x)$. $R_x(t)$ represents the value of thread t's logical clock when it last read from x. For a location x and thread t: $R_x(t) = 0$ means that t has never read from x. Otherwise $R_x(t)$ was t's logical clock last time t read from x
- $W : Locations \to VC$. Similar to R.

Initial analysis state: $C = (inc_0(\bot), inc_1(\bot), \ldots, inc_{N-1}(\bot))$. Thread t knows its logical clock is 1 and thinks all other threads' logical clocks are 0. $L = \lambda m.\ \bot$. No thread has unlocked any mutex. $R = \lambda x.\ \bot$. $W = \lambda x.\ \bot$. No thread has read from or written to any location.

ACQUIRE:

$$\frac{C' = C[t \mapsto (C_t \sqcup L_m)]}{(C,L,R,W) \to_{acq(t,m)} (C',L,R,W)}$$

READ:

$$\frac{W_x \sqsubseteq C_t \quad R' = R[x \mapsto R_x[t \mapsto C_t(t)]]}{(C,L,R,W) \to_{rd(t,x)} (C,L,R',W)}$$

WRITE:

$$\frac{W_x \sqsubseteq C_t \quad R_x \sqsubseteq C_t \quad W' = W[x \mapsto W_x[t \mapsto C_t(t)]]}{(C,L,R,W) \to_{wr(t,x)} (C,L,R,W')}$$

RELEASE:

$$\frac{L' = L[m \mapsto C_t] \quad C' = C[t \mapsto inc_t(C_t)]}{(C,L,R,W) \to_{rel(t,m)} (C',L',R,W)}$$

$$\frac{\exists u. W_x(u) > C_t(u)}{(C,L,R,W) \to_{rd(t,x)} WriteReadRace(u,t,x)}$$

$$\frac{\exists u. W_x(u) > C_t(u)}{(C,L,R,W) \to_{wr(t,x)} WriteWriteRace(u,t,x)}$$

$$\frac{\exists u. R_x(u) > C_t(u)}{(C,L,R,W) \to_{wr(t,x)} ReadWriteRace(u,t,x)}$$

Theoretic

Introduction

Formal Properties in asynchronous computation:

• **Safety: Nothing bad happens ever.** If it is violated, it is done by a **finite** computation.

This is a safety property as ... violates the property and can be done in **finite** time. The "bad thing" is ....

• **Liveness: Something good happens eventually.** Cannot be violated by a finite computation (intuition: we can always run longer and see what happens).

This can be read as "... will **eventually** ...". This is a liveness property as one cannot violate it in a **finite** number of steps: If ... yet, they may do so eventually at a later time. The "good thing" is ....

Amdahl's Law:

$$Speedup = \frac{1\text{-thread execution time}}{n\text{-thread execution time}} = \frac{1}{1-p+\frac{p}{n}}.$$

p: parallel fraction. n: number of threads. (We need very high parallel fraction to achieve high speedup)

Basics of Shared-Memory Concurrency

Read-Modify-Write (RMW):

• Holds some value x

• Has a method *getAndMumble()* such that: Returns object's prior value x. Replaces x with *mumble(x)* for some function *mumble*. And it does so **atomically**, in a mutually exclusive way.

Weak RMW: enables synchronisation between **two** threads. (E.g. exchange – xchg, FetchAndAdd-FAA)

Strong RMW: enables synchronisation between an **arbitrary** number of threads. (E.g. CompareAndSet-CAS)

Concurrent Semantics: Sequential Consistency (SC)

Sequential Consistency (SC) (=interleaving semantics): The instructions of each thread are executed in order. Instructions of different threads can be interleaved arbitrarily.

Notations:

$x, y, z, \ldots$	**Shared memory locations**
$a, b, c, \ldots$	**Private (thread-local) registers**
$E, E_1, \ldots$	Expressions over values (integers) and registers
$a \coloneqq x$	**Read** from location x into register a
$x \coloneqq a$	**Write** contents of register a to location x
$a \coloneqq E$	**Assignment**: compute E and write it to a

$CONWHILE$ sequential commands:

$C \in Com ::= a \coloneqq E$ [assignment]
$\mid a \coloneqq x$ [(memory) read]
$\mid x \coloneqq a$ [(memory) write]
$\mid a \coloneqq CAS(x, E, E) \mid FAA(x, E)$ [(memory) RMWs]
$\mid skip \mid C; C \mid while\ B\ do\ C$
$\mid if\ B\ then\ C\ else\ C$

$CONWHILE$ concurrent programs: $P \in Prog \triangleq Tid \rightarrow Com$

For n-threaded program P: $C_1 || C_2 || \ldots || C_n$ with $dom(P) = \{\tau_1, \ldots, \tau_n\}$ and $P(\tau_i) = C_i$ for $i \in \{1, \ldots, n\}$.

SC Configurations:

• **Shared memory:** $M \in Mem \triangleq Loc \rightarrow Val.\ Val$: set of all values, including integer and Boolean values.

• **Store:** $s \in Store \triangleq Reg \rightarrow Val.$

• **Store Map:** $S \in SMap \triangleq Tid \rightarrow Store.\ s = S(\tau)$: the store map of τ. $s(b)$: the value of register b for thread τ.

• **SC configuration:** (P, S, M)

SC Operational Semantics:

Program transitions describe the steps in program executions, e.g. how a conditional (if) statement is executed.

Storage transitions describe how instructions interact with the storage (memory) system, e.g. how a memory read behaves.

• If the program takes a silent transition:

$$\frac{P, S \xrightarrow{\tau:\epsilon}_p P', S'}{P, S, M \rightarrow P', S', M}$$

• If both the program and storage systems take the same transition, then we can combine them:

$$\frac{P, S \xrightarrow{\tau:l}_p P', S' \qquad M \xrightarrow{\tau:l}_m M'}{P, S, M \rightarrow P', S', M'}$$

SC Transition Labels:

• **empty label ϵ:** a silent transition (e.g. when reducing skip;C to C);

• **read label (R, x, v):** reading value v from memory location x

• **write label (W, x, v):** writing value v to memory location x

• **successful RMW label (U, x, v_o, v_n):** updating the value of location x to v_n when the old value of x in memory is v_o

• **failed RMW label $(U, x, v_o, \bot)$:** a failed CAS instruction where the old value of x in memory, v_o, does not match the expected value.

SC Sequential Transitions: $C, s \xrightarrow{l}_c C', s'$.

$$\frac{s(a) = v}{x \coloneqq a, s \xrightarrow{(\mathtt{W},x,v)}_c \mathtt{skip}, s} \qquad \frac{s' = s[a \mapsto v]}{a \coloneqq x, s \xrightarrow{(\mathtt{R},x,v)}_c \mathtt{skip}, s'}$$

$$\frac{\mathrm{eval}(s, E) = v \qquad v_n = v_o + v \qquad s' = s[a \mapsto v_o]}{a \coloneqq \mathtt{FAA}(x, E), s \xrightarrow{(\mathtt{U},x,v_o,v_n)}_c \mathtt{skip}, s'}$$

$$\frac{\mathrm{eval}(s, E_e) = v_e \qquad v_e = v_o \qquad \mathrm{eval}(s, E_n) = v_n \qquad s' = s[a \mapsto 1]}{a \coloneqq \mathtt{CAS}(x, E_e, E_n), s \xrightarrow{(\mathtt{U},x,v_o,v_n)}_c \mathtt{skip}, s'}$$

$$\frac{\mathrm{eval}(s, E_e) = v_e \qquad v_e \neq v_o \qquad s' = s[a \mapsto 0]}{a \coloneqq \mathtt{CAS}(x, E_e, E_n), s \xrightarrow{(\mathtt{U},x,v_o,\bot)}_c \mathtt{skip}, s'}$$

Program transition:

$$\frac{P(\tau)=C \qquad S(\tau)=s \qquad C, s \xrightarrow{l}_c C', s' \qquad P'=P[\tau \mapsto C'] \qquad S'=S[\tau \mapsto s']}{P, S \xrightarrow{\tau:l}_p P', S'}$$

SC Storage Transitions: $M \xrightarrow{\tau:l}_m M'$

$$\frac{M(x) = v}{M \xrightarrow{\tau:(\mathtt{R},x,v)}_m M} \qquad \frac{M' = M[x \mapsto v]}{M \xrightarrow{\tau:(\mathtt{W},x,v)}_m M'} \qquad \frac{M(x) = v_o}{M \xrightarrow{\tau:(\mathtt{U},x,v_o,\bot)}_m M}$$

$$\frac{M(x) = v_o \qquad M' = M[x \mapsto v_n]}{M \xrightarrow{\tau:(\mathtt{U},x,v_o,v_n)}_m M'}$$

Many Steps of Evaluation: $\rightarrow^*$ for the reflexive transitive closure of $\rightarrow$: $P, S, M \rightarrow^* P', S', M' \Leftrightarrow (P, S, M) = (P', S', M') \vee \exists P'', S'', M''. P, S, M \rightarrow P'', S'', M'' \wedge P'', S'', M'' \rightarrow^* P', S', M'$.

SC Traces:

Initial Memory: $M_0 \triangleq \lambda x. 0$. **Initial Store:** $s_0 \triangleq \lambda a. 0$. **Initial Store Map:** $S_0 \triangleq \lambda\tau. s_0$. **Terminated Program:** $P_{skip} \triangleq \lambda\tau. skip$.

Initial SC-configuration: (P, S_0, M_0).

SC-trace: an evaluation path that starts from the initial SC-configuration of P and terminates with P_{skip}. i.e. there exists S and memory M such that: $P, S_0, M_0 \rightarrow^* P_{skip}, S, M$.

A pair (S, M) denotes an **SC-outcome** iff $P, S_0, M_0 \rightarrow^* P_{skip}, S, M$.

SC is **not deterministic nor confluent.**

Deterministic:

$\forall P, S, M, P_1, S_1, M_1, P_2, S_2, M_2.$
if $\quad P, S, M \rightarrow P_1, S_1, M_1 \wedge P, S, M \rightarrow P_2, S_2, M_2$
then $\ (P_1, S_1, M_1) = (P_2, S_2, M_2)$

Confluent:

$\forall P, S, M, P_1, S_1, M_1, P_2, S_2, M_2.$
if $\quad P, S, M \rightarrow^* P_1, S_1, M_1 \wedge P, S, M \rightarrow^* P_2, S_2, M_2$
then $\exists P', S', M'.\ P_1, S_1, M_1 \rightarrow^* P', S', M' \wedge P_2, S_2, M_2 \rightarrow^* P', S', M'$

Concurrent Semantics: Total Store Ordering (TSO)

Weak Memory Models (WMM):

• Unlike SC, the instructions of a thread may be **reordered**, i.e. they may execute **out of order**

• May observe **weak behaviours** (anomalies) not observable under SC.

TSO (Total Store Ordering)

• Allows the following for **adjacent** instructions (in program order):

1. **write-read reordering on different locations:** a later read on y can be reordered before an earlier write on x when $x \neq y$.

2. **write-read reordering on same location with write forwarding:** a later read on x can be reordered before an earlier write on x provided that the value of the write is forwarded to (inlined in) the read.

• 1 leads to **weak SB (store buffering)**; 2 alone **does not cause weak behaviours**.

• No other reordering or change is allowed

• i.e. TSO = SC + write-read reordering on different locations + write-to-read on same location + write forwarding

For TSO, we extend the $CONWHILE$ sequential commands:

$C \in Com ::= mfence$ [memory fence]

TSO Configurations:

• **Buffer**, as a FIFO sequence of delayed write labels: $b \in Buff \triangleq Seq < WLab >. WLab \triangleq \{(W, x, v) | x \in Loc \wedge v \in Val\}$.

• **Buffer Map:** $B \in BMap \triangleq Tid \rightarrow Buff.\ b = B(\tau)$: the buffer of τ.

• **TSO Configuration:** (P, S, M, B).

TSO Transition Label: either an SC label, or a memory fence label MF for executing an $mfence$.

TSO sequential transitions include all SC sequential transitions, as well as:

$$\frac{}{\mathtt{mfence}, s \xrightarrow{\mathtt{MF}}_c \mathtt{skip}, s}$$

TSO Storage transitions: $M, B \xrightarrow{\tau:l}_m M', B'$.

$$\frac{B(\tau) = b \qquad b' = b.(\mathtt{W}, x, v) \qquad B' = B[\tau \mapsto b']}{M, B \xrightarrow{\tau:(\mathtt{W},x,v)}_m M, B'}$$

$$\frac{B(\tau) = b \qquad \mathrm{get}(M, b, x) = v}{M, B \xrightarrow{\tau:(\mathtt{R},x,v)}_m M, B}$$

$$\mathrm{get}(M, b, x) \triangleq \begin{cases} v & \text{if } \exists b_1, b_2.\ b = b_1.(\mathtt{W}, x, v).b_2 \\ & \quad \wedge \neg\exists v'.\ (\mathtt{W}, x, v') \in b_2 \\ M(x) & \text{otherwise} \end{cases}$$

$$\frac{B(\tau) = \emptyset}{M, B \xrightarrow{\tau:\mathtt{MF}}_c M, B} \qquad \frac{B(\tau) = \emptyset \qquad M(x) = v_o}{M, B \xrightarrow{\tau:(\mathtt{U},x,v_o,\bot)}_m M, B}$$

$$\frac{B(\tau) = \emptyset \qquad M(x) = v_o \qquad M' = M[x \mapsto v_n]}{M, B \xrightarrow{\tau:(\mathtt{U},x,v_o,v_n)}_m M', B}$$

$$\frac{B(\tau) = (\mathtt{W}, x, v).b \qquad M' = M[x \mapsto v] \qquad B' = B[\tau \mapsto b]}{M, B \xrightarrow{\tau:\epsilon}_m M', B'}$$

Initial Buffer Map: $B_0 \triangleq \lambda\tau. \emptyset$. **Initial TSO-configuration:** (P, S_0, M_0, B_0).

TSO-trace: an evaluation path that starts from the initial TSO-configuration of P and terminates with P_{skip} and empty buffers; i.e. there exists S and memory M such that: $P, S_0, M_0, B_0 \rightarrow^* P_{skip}, S, M, B_0$.

A pair (S, M) denotes an **TSO-outcome** iff $P, S_0, M_0, B_0 \rightarrow^* P_{skip}, S, M, B_0$.

TSO is **not deterministic nor confluent.**

NOTE: *TSO* allows for sfence (with SF label); and sfence-read reordering. The **buffer** for memory transitions can hold SF labels.

NOTE: *PSO* is like *TSO* but allows write-write reordering on different memory locations.

Declarative Semantics

$\mathtt{typ}((\mathtt{R}, x, v_r)) \triangleq \mathtt{R} \quad \mathtt{typ}((\mathtt{W}, x, v_w)) \triangleq \mathtt{W} \quad \mathtt{typ}((\mathtt{U}, x, v_r, v_w)) \triangleq \mathtt{U}$

$\mathtt{val_r}((\mathtt{R}, x, v_r)) \triangleq v_r \qquad \mathtt{val_r}((\mathtt{U}, x, v_r, v_w)) \triangleq v_r$

$\mathtt{val_w}((\mathtt{W}, x, v_w)) \triangleq v_w \qquad \mathtt{val_w}((\mathtt{U}, x, v_r, v_w)) \triangleq v_w$

$\mathtt{loc}((\mathtt{R}, x, v_r)) = \mathtt{loc}((\mathtt{W}, x, v_w)) = \mathtt{loc}((\mathtt{U}, x, v_r, v_w)) \triangleq x$

$[A]$ for the identity relation on A: $[A] \triangleq \{(a, a) | a \in A\}$

$r; r'$ for composition

$r^?$ for the reflexive closure of r

$\mathrm{irreflexive}(r) \overset{def}{\Longleftrightarrow} \neg\exists a.\ (a, a) \in r$

$\mathrm{acyclic}(r) \overset{def}{\Longleftrightarrow} \mathrm{irreflexive}(r^+)$

$A_\tau \triangleq \{a \in A | \mathtt{tid}(a) = \tau\}$ and $r_\tau \triangleq r \cap (A_\tau \times A_\tau)$

ri for internal (same-thread) r edges:

$ri \triangleq \{(a, b) \in r | \mathtt{tid}(a) = \mathtt{tid}(b) \vee \mathtt{tid}(a) = 0\}$

re for external r edges: $re \triangleq r \setminus ri$

$A_x \triangleq \{a \in A | \mathtt{loc}(a) = x\}$ and $r_x \triangleq r \cap (A_x \times A_x)$

$r|_{\mathtt{loc}} \triangleq \{(a, b) \in r | \mathtt{loc}(a) = \mathtt{loc}(b)\}$

$G.\mathrm{E} \triangleq E$

$G.\mathrm{po} \triangleq po$

$G.\mathtt{rf} \triangleq rf$

$G.\mathrm{R}_x \triangleq G.\mathrm{R} \cap \{r \in E \mid \mathtt{loc}(r) = x\}$

rf = "read from" $WU \rightarrow R$

po = "program order" $E \rightarrow E$

mo = "modification order" $WU \rightarrow WU$

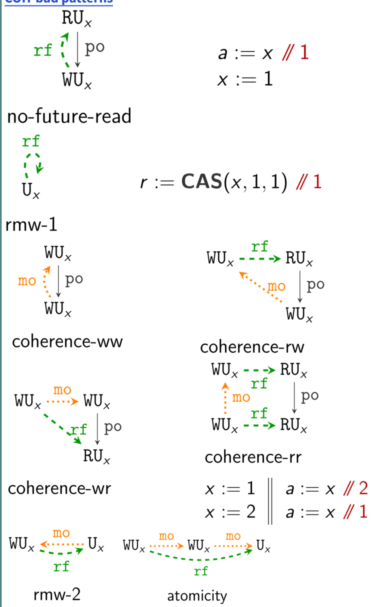
rb = "read before" $R \rightarrow WU$

ppo = "preserved program order" $E \rightarrow E$

hb = "happened before" $E \rightarrow E$

sw = "synchronizes with" $E \rightarrow E$

COH-bad patterns



In coherent executions, an RMW event may only read from its immediate mo-predecessor.

RA needed

Spinlock implementation

lock(l) :	unlock(l) :
$r := 0$; **while** $\neg r$ **do** $\quad r :=$ **CAS**$(l, 0, 1)$	$l := 0$

You need RA to implement locks (COH too weak); "synchronizing" every write & read.

C11 semantics

Reads: (R^m, x, v), where $m \in \{\mathbf{na}, \mathbf{rlx}, \mathbf{acq}, \mathbf{sc}\}$

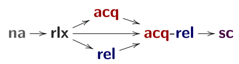
Writes: (W^m, x, v), where $m \in \{\mathbf{na}, \mathbf{rlx}, \mathbf{rel}, \mathbf{sc}\}$

RMWs: $(\mathrm{U}^m, x, v_o, v_n)$ or $(\mathrm{U}^m, x, v_o, \bot)$, where $m \in \{\mathbf{rlx}, \mathbf{acq}, \mathbf{rel}, \mathbf{acq\text{-}rel}, \mathbf{sc}\}$

Fences: F^m, where $m \in \{\mathbf{acq}, \mathbf{rel}, \mathbf{acq\text{-}rel}, \mathbf{sc}\}$

Given an event e, we write $\mathtt{mod}(e)$ for the mode of e.

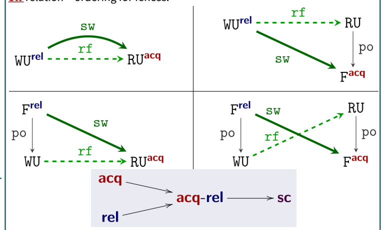
"Mode strength" order $\sqsubset$ is given by:



Given a set of events E, and a mode m, we write:

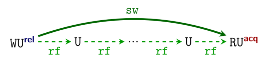
$$E^{\sqsupseteq m} \triangleq \{e \in E \mid m = \mathtt{mod}(e) \vee m \sqsubset \mathtt{mod}(e)\}$$

sw relation + ordering for fences:



Release sequences (RMW's):

$x_{\mathrm{rlx}} := 42;$ $y_{\mathrm{rel}} := 1$	$a := \mathbf{FAI}_{\mathrm{rlx}}(y)$; // 1	$b := y_{\mathrm{acq}}$; // 2 $c := x_{\mathrm{rlx}}$; // 0



$$\mathtt{sw} \triangleq ([\mathtt{WU}^{\sqsupseteq \mathbf{rel}}] \cup [\mathtt{F}^{\sqsupseteq \mathbf{rel}}]; \mathtt{po}); \mathtt{rf}^+; ([\mathtt{RU}^{\sqsupseteq \mathbf{acq}}] \cup \mathtt{po}; [\mathtt{F}^{\sqsupseteq \mathbf{acq}}])$$

Concurrent Objects

Concurrent Objects

Invocation Notation: *thread object.method(arguments)*. E.g. $A\ q.enq(x)$

Response Notation: *thread object: result*. E.g. $A\ q: void$

An invocation is **pending** if it has no matching response.

Sequential Histories: Method calls of different threads do not interleave

Well-Formed Histories: Per-thread projections sequential

Equivalent Histories: Per-thread projections are the same.

A method call **precedes** another if response event precedes invocation event.

Notation: Given History H, method executions m_0 and m_1 in H

• We say $m_0 \rightarrow_H m_1$, if m_0 precedes m_1

• Relation $m_0 \rightarrow_H m_1$ is a Partial order OR Total order if H is sequential.

A sequential (multi-object) history H is **legal** if **for every object x, $H|x$ is in the sequential spec for x.** (per-object projection)

History H is **linearizable** if it can be extended to G by

– Appending zero or more responses to pending invocations

– Discarding other pending invocations

• So that G is **equivalent** to

– **Legal sequential** history S

– where $\rightarrow_G \subseteq \rightarrow_S$

Composability Theorem: History H is linearizable if and only if for every object x, $H|x$ is linearizable.

History H is **Sequentially Consistent** if it can be extended to G by

– Appending zero or more responses to pending invocations

– Discarding other pending invocations

• So that G is **equivalent** to a

– **Legal sequential** history S

Sequential Consistency: No need to preserve real-time order

• Cannot **re-order operations done by the same thread**

• Can **re-order non-overlapping operations done by different threads (only keep relative order in the same thread)**

• Not composable

Proof H is Linearizable:

If multiple objects: Given the composability of linearizability, it suffices to show that for each object x, $H|x$ is linearizable. Proof each with steps:

Represent history H formally as: ... Let G denote the following complete history, and S denote the following sequential history.

Note that: G is obtained from H by discarding the pending invocation ... / appending the matching response ... for the pending invocation ...

We then have: $G|A = S|A = \cdots$, $G|B = S|B = \cdots$, ... and thus we know: G and S are equivalent. (1)

With the following S_p and S_q: ... Since for object p, S_p is in the sequential spec for p, and similarly for object q, S_q is in the sequential spec for q, we can show that:

S is a legal, sequential history of memory location objects. (2)

From the definition of $\rightarrow$ we have: $\rightarrow_G = \cdots$, $\rightarrow_S = \cdots$ and thus $\rightarrow_G \subseteq \rightarrow_S$. (3)

From (1), (2), (3) we can conclude that H is linearizable.

Proof H is Sequentially Consistent: same as Proof H is Linearizable expect no need to proof (3).

Proof H *is* NOT Linearizable: Proof by contradiction. Assume H is linearizable. As H is a complete history (without pending invocations), this means there exists a history S such that:

(a) S is equivalent to H;

(b) S is a legal sequential history; and

(c) $\rightarrow_H \subseteq \rightarrow_S$

As H is a sequential history and thus $\rightarrow_H$ is total, from (c) we know that $S = H$. On the other hand, we know H is not a legal history (because ...), and thus (since $S = H$) we know S is not a legal history. This, however, leads to a contradiction as in (b) we assumed that S is a legal history.

Proof H is NOT Sequentially Consistent: Proof by contradiction. Assume H is linearizable. As H is a complete history (without pending invocations), this means there exists a history S such that:

(a) S is equivalent to H;

(b) S is a legal sequential history.

For S to be a legal history as per (b) we must have: ... (1)

From (a) and definition of H we have: ... (2)

From (1) and (2) we have: ...

That is ... and thus $\rightarrow_S$ contains a cycle. This leads to a contradiction as S is a sequential history and thus $\rightarrow_S$ must be a strict (irreflexive) total order.

Progress Conditions:

• Deadlock-free: some thread trying to acquire the lock eventually succeeds.

• Starvation-free: every thread trying to acquire the lock eventually succeeds.

• Lock-free: some thread calling a method eventually returns.

• Wait-free: every thread calling a method eventually returns.

	Non-Blocking	Blocking
Everyone makes progress	Wait-Free	Starvation-Free
Someone makes progress	Lock-Free	Deadlock-Free